Database Schema & Architecture Design

ProjectsHub (Monograph & Software Projects Management Platform)

Document Version: 1.0 **SDLC Phase:** 2

1. Schema Design Approach (Document-Oriented Architecture)

MongoDB serves as the primary data store for ProjectsHub due to its flexible document schema, native support for embedded document arrays (enabling inline team structures, audit trails, and tag sets), and high-performance text and compound indexing for the public storefront.

Core Architectural Principles

- **Multi-Tenancy Isolation:** Every top-level collection includes an `institutionId` reference, facilitating seamless multi-tenant deployments and future open-source expansion across multiple universities.
- **Hybrid Embedding & Referencing:** Frequently read co-located data (such as project members, monograph metadata, and software links) is embedded directly within the primary `projects` document to eliminate costly join operations. External references (`ObjectId`) are reserved for structural entities like faculties, departments, and user accounts.
- **Atomic Versioning & Immutability:** Version identifiers (`v1`, `v2`) and approval locks are maintained directly on project records to ensure transactional integrity and prevent concurrent modifications post-approval.

---

2. Collections & Schema Definitions

Collection 1: `universities`

Stores top-level institutional records for multi-tenant management.

Field	Type	Description / Constraints
`_id`	ObjectId	Primary Key
`name`	String	Official institution name (e.g., `"Kandahar University"`)
`domain`	String	Institutional domain identifier (e.g., `"kdun.edu.af"`)
`createdAt`	ISODate	Record creation timestamp

Collection 2: `faculties`

Defines academic faculties and their constituent departments.

Field	Type	Description / Constraints
`_id`	ObjectId	Primary Key
`institutionId`	ObjectId	Reference to `universities`
`name`	String	Faculty name (e.g., `"Faculty of Computer Science"`)
`departments`	Array [Object]	List of nested objects: `[{ _id: ObjectId, name: String }]`
`createdAt`	ISODate	Record creation timestamp

Collection 3: `users`

Unified user collection covering administrative staff, faculty members, and students.

Field	Type	Description / Constraints
`_id`	ObjectId	Primary Key
`institutionId`	ObjectId	Reference to `universities`
`facultyId`	ObjectId	Reference to `faculties`
`departmentId`	ObjectId	Reference to specific department within faculty
`studentId`	String	Unique student registration identifier (e.g., `"2019-CS-042"`)
`fullName`	String	Full user name
`email`	String	Unique email address
`passwordHash`	String	Securely hashed credential
`role`	Enum	`"super_admin"` | `"faculty_dean"` | `"teacher"` | `"student"`
`mustChangePassword`	Boolean	Default: `true`
`createdAt`	ISODate	Account creation timestamp

- **Indexes:** `{ studentId: 1, institutionId: 1 }` (Unique), `{ email: 1 }` (Unique)

Collection 4: `technology_tags`

Central taxonomy of technology stack tags managed by platform administrators.

Field	Type	Description / Constraints
`_id`	ObjectId	Primary Key
`name`	String	Tag display name (e.g., `"React Native"`)
`category`	String	Functional category (e.g., `"Frontend Framework"`)
`slug`	String	URL-friendly slug (e.g., `"react-native"`)

Collection 5: `projects` (Primary Core Collection)

Main collection managing lifecycle submissions, embedded deliverables, team structures, review logs, and approval locking.

Field	Type	Description / Constraints
`_id`	ObjectId	Primary Key
`institutionId`	ObjectId	Reference to `universities`
`facultyId`	ObjectId	Reference to `faculties`
`departmentId`	ObjectId	Reference to department
`graduationYear`	Number	Academic year (e.g., `2026`)
`type`	Enum	`"universal"` | `"science"`
`title`	String	Project submission title
`status`	Enum	`"draft"` | `"submitted"` | `"revision_requested"` | `"approved"` | `"rejected"`

Field	Type	Description / Constraints
`versionNumber`	Number	Incremental version counter (Default: 1)
`isLocked`	Boolean	Edit lock state (Default: `false`)
`leader`	Object	Embedded document: `{ userId, studentId, fullName }`
`members`	Array [Object]	List of embedded objects: `[{ userId, studentId, fullName }]`
`monograph`	Object	Embedded monograph details (see below)
`software`	Object	Embedded software details (see below)
`assignedTeacherId`	ObjectId	Reference to assigned reviewer (`users`)
`reviews`	Array [Object]	Historical log of review decisions (see below)
`createdAt`	ISODate	Submission initialization timestamp
`updatedAt`	ISODate	Last modification timestamp

Embedded Sub-Document Schemas (`projects`)

- **`monograph` Object:**
  - `pdfFileUrl`: String (Cloud storage target URL)
  - `abstract`: String (Executive summary / synopsis)
  - `keywords`: Array of Strings

    - `defenseDate`: ISODate
    - `citationCount`: Number
- **`software` Object:**
    - `githubRepoUrl`: String
    - `liveDemoUrl`: String
    - `screenshots`: Array of Strings (Asset storage URLs)
    - `techTagIds`: Array of ObjectIds (References to `technology_tags`)
- **`reviews` Array Items:**
    - `_id`: ObjectId
    - `reviewerId`: ObjectId (Reference to `users`)
    - `action`: Enum (`"approve"` | `"reject"` | `"request_revision"`)
    - `feedbackText`: String
    - `versionAtReview`: Number
    - `createdAt`: ISODate

Collection 6: `notifications`

System notification records generated upon state changes (e.g., project approval or rejection).

Field	Type	Description / Constraints
`_id`	ObjectId	Primary Key
`userId`	ObjectId	Target recipient (`users`)
`projectId`	ObjectId	Associated project context (`projects`)
`type`	Enum	`"ACCEPTED"` | `"REJECTED"`
`title`	String	Notification subject
`message`	String	Detailed message content

Field	Type	Description / Constraints
`isRead`	Boolean	Delivery/read status (Default: `false`)
`createdAt`	ISODate	Dispatch timestamp

---

3. MongoDB Indexing Strategy for Public Storefront Search

To maintain sub-millisecond query performance across public catalog filtering and administrative views, the following indexes are established on the `projects` collection:

1. Full-Text Search Index

Supports natural language search across titles, abstracts, and key terminology.t
db.projects.createIndex({
"title": "text",
"monograph.abstract": "text",
"monograph.keywords": "text"
})

2. Faceted Filtering Compound Index

Optimizes multi-field catalog queries on the public storefront.

```javascript

db.projects.createIndex({

  "status": 1,

  "graduationYear": -1,

  "facultyId": 1,
```

```
  "departmentId": 1,
  "software.techTagIds": 1
})
```

3. Student Dashboard Query Index

Accelerates portal dashboard lookups for individual student submissions.db.projects.createIndex({

```
  "leader.userId": 1,
  "members.userId": 1
})
```

---

4. Key MongoDB Workflow Patterns

Atomic Project Locking

When an authorized reviewer issues an approval, an atomic conditional update locks the record against concurrent edits:db.projects.updateOne(

```
  { _id: projectId, isLocked: false },
  { $set: { status: "approved", isLocked: true } }
)
```

If `isLocked` is already `true`, the operation updates zero documents, allowing the API layer to reject secondary modification attempts immediately.

Revision Versioning (v1 → v2)

When revisions are requested, the platform either pushes the active state to a dedicated project_versions collection or increments the versionNumber counter while appending prior iterations to an internal versionHistory array.

Efficient Team Membership Queries

Because team members are embedded within the project document, retrieving all projects linked to a specific student requires a single query with an $or condition:db.projects.find({

```
  $or: [
    { "leader.userId": targetUserId },
    { "members.userId": targetUserId }
  ]
})
```